PIG Documentation:

- Introduction to Apache Pig
- Building Blocks (Bag, Tuple & Field)
- Installing Pig
- Different modes of execution of PIG
- Pig Latin
- Data types
- Working with various PIG Commands covering all the functions in PIG
- Developing PIG scripts
- Parameter Substitution
- Passing parameters though a param file
- Joins (Left Outer, Right Outer, Full Outer)
- Nested queries
- Specialized joins in PIG (Replicated, Skewed & Merge Join)
- HCatalog(Getting data from hive to pig & vice versa)
- Working with un-structured data
- Working with Semi-structured data like XML, JSON
- Hands-On Exercise
- Assignment on PIG

Introduction to Apache Pig:

Apache Pig is a tool used to analyze large amounts of data by represeting them as data flows. Using the PigLatin scripting language operations like ETL (Extract, Transform and Load), adhoc data anlaysis and iterative processing can be easily achieved.

Pig is an abstraction over MapReduce. In other words, all Pig scripts internally are converted into Map and Reduce tasks to get the task done. Pig was built to make programming MapReduce applications easier. Before Pig, Java was the only way to process the data stored on HDFS.

Pig was first built in Yahoo! and later became a top level Apache project. In this series of we will walk through the different features of pig using a sample dataset.

Bag, Tuple & Field:

- A relation is a bag (more specifically, an outer bag).
- Bag {(12.5,hello world,-2),(2.87,bye world,10)}

A bag is a collection of tuples. A bag is an unordered collection of tuples. A relation is a special kind of bag, sometimes called an outer bag. An inner bag is a bag that is a field within some A bag is represented by tuples separated by commas, all enclosed by curly

Installing Pig:

Sreeni Docs

→Download Pig

$ wget http://mirror.symnds.com/software/Apache/pig/pig-0.12.0/pig-0.12.0.tar.gz

→Untar

$ tar xvzf pig-0.12.0.tar.gz

→Rename to folder for easier access:

$ mv pig-0.12.0 pig

→Update .bashrc to add the following:

export PATH=$PATH:/home/hduser/pig/bin

Different modes of execution of PIG:

Pig can be started in one of the following two modes:

Local Mode

Cluster Mode

Using the '-x local' options starts pig in the local mode whereas executing the pig command without any options starts in Pig in the cluster mode. When in local mode, pig can access files on the local file system. In cluster mode, pig can access files on HDFS.

Restart your terminal and execute the pig command as follows:

To start in Local Mode:

$ pig -x local

2013-12-25 20:16:26,258 [main] INFO org.apache.pig.Main - Apache Pig version 0.12.0 (r1529718) compiled Oct 07 2013, 12:20:14

2013-12-25 20:16:26,259 [main] INFO org.apache.pig.Main - Logging error messages to: /home/hduser/pig/myscripts/pig_1388027786256.log

2013-12-25 20:16:26,281 [main] INFO org.apache.pig.impl.util.Utils - Default bootup file /home/hduser/.pigbootup not found

2013-12-25 20:16:26,381 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: file:///

grunt>

To start in Cluster Mode:

$ pig

Sreeni Docs

2013-12-25 20:19:42,274 [main] INFO org.apache.pig.Main - Apache Pig version 0.12.0 (r1529718) compiled Oct 07 2013, 12:20:14

2013-12-25 20:19:42,274 [main] INFO org.apache.pig.Main - Logging error messages to: /home/hduser/pig/myscripts/pig_1388027982272.log

2013-12-25 20:19:42,300 [main] INFO org.apache.pig.impl.util.Utils - Default bootup file /home/hduser/.pigbootup not found

2013-12-25 20:19:42,463 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:54310

2013-12-25 20:19:42,672 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to map-reduce job tracker at: hdfs://localhost:9001

grunt>

This command presents you with a grunt shell. The grunt shell allows you to execute PigLatin statements to quickly test out data flows on your data step by step without having to execute complete scripts. Pig is now installed and we can go ahead and start using Pig to play with data.

Pig Latin:

Pig Latin statements are the basic constructs you use to process data using Pig. A Pig Latin statement is an operator that takes a relation as input and produces another relation as output.

Data types:

Int, Float, chararray, bytearray, double, datetime

Working with various PIG Commands covering all the functions in PIG:

Touple = Record

Bag = Table

==========

pig -x local -->(Local)

pig -->(MR mode)

grunt>

Sreeni Docs

Load Bag with full schema:

A = load '/root/mydir/files_dir/emp.txt' using PigStorage(',') as (eid:int,ename:chararray,desg:chararray,sal:double,dname:chararray,did:int);

Load Bag without schema:

A = load '/root/mydir/emp.txt'; // [default delimeter is TabSpace.]

Load Bag without semi schema: [Delimiter should be match with the delimeter exist inside the file]

A = load '/root/mydir/emp.txt' using PigStorage(','); // without field names [default column names are $0,$1,$2,,,]

or

A = load '/root/mydir/emp.txt' using PigStorage(',') as (eid,ename,dept,sal,dname,did); // without data types [default data type = Bytearray which allows char's, numbers]

or

A = load '/root/mydir/emp.txt' using PigStorage(',') as (eid:int,ename,dept,sal,dname,did:int); // few columns without data type

Store a Bag: [storing location should be always a new folder. It will create a new file under the new folder with name part-m-00000. We can give any delimiter for storing. Default delimiter is the delimeter given while creation.]

store A into '/tmp/samp1' using PigStorage('$'); // it will store under /tmp/samp1/part-m-00000

view the file:

cat /tmp/samp1/part-m-00000

==========FILTER==========

B = filter A by sal > 1500;

or

B = filter A by $3 > 1500;

==========LIMIT==========

Sreeni Docs

Get top 5 tupels:

```
C = limit B 5;

dump C
```

==========ORDER==========

```
B = order A by sal; //[ Default order is Ascending ]

C =order A by sal desc;
```

==========DISTINCT==========

```
A = load '/root/mydir/emp.txt';

B = foreach A generate $2;

C = DISTINCT B;

dump C
```

Developing PIG scripts and and Parameter Substitution:

```
pig -x local '/root/mydir/files_dir/emp.txt'

---------------------------------------------------------------------------

A =load '/root/mydir/files_dir/emp.txt' using PigStorage(',') as
(eid:int,ename:chararray,desg:chararray,sal:double,dname:chararray,did:int);

sal_above_1500 = filter A by sal > 1500;

five_tupels = limit sal_above_1500 5;

order_asc = order five_tupels by sal;

order_dsc = order order_asc by sal desc;

desg_bag = foreach order_dsc generate $2;

dist_desg = distinct desg_bag;

store order_dsc into '/root/mydir/pgout/pgout5' using PigStorage('$');


pig -x local '/home/training/RamLocal/PIG/pig_scripts/pg_script1.pig'
```

Sreeni Docs

```
A = load '/home/training/RamLocal/PIG/pig_data/emp.txt' using PigStorage(',') as
(eid:int,ename:chararray,desg:chararray,sal:double,dname:chararray,did:int);

B = filter A by sal > 1500;

C = order A by sal;

D = order A by sal desc;

E = foreach D generate $2;

F = distinct E;

store F into '/home/training/RamLocal/PIG/pig_out/pig_out2' using PigStorage('|');
```

==========
==

Working with un-structured data:

```
F = load '/practice/ram/' as(line:chararray);

D = foreach F generate TOKENIZE($0) as token;

E = foreach D generate FLATTEN($0) as flat;

K = group E by $0;

countwords = foreach K GENERATE $0,COUNT($1);

counts = order countwords by $1 desc;

last = limit counts 1;

dump last;
```

==========
==

Working with Semi-structured data like XML:

Register Jar files:

```
grunt> REGISTER /usr/lib/pig/piggybank.jar

 A = load '/usr/local/hadoop/conf/hdfs-site.xml' using
org.apache.pig.piggybank.storage.XMLLoader('property') as (x:chararray);

 B = foreach A generate
FLATTEN(REGEX_EXTRACT_ALL(x,'<property>\\s*<name>(.*)</name>\\s*<value>(.*)</value>\\s*
</property>'))
```


Sreeni Docs

HCatalog:

HCatalog opens up the hive metadata to other mapreduce tools. Every mapreduce tools has its own notion about HDFS data (example Pig sees the HDFS data as set of files, Hive sees it as tables). With having table based abstraction, HCatalog supported mapreduce tools do not need to care about where the data is stored, in which format and storage location (HBase or HDFS).

Ex:

A = LOAD 'student' USING org.apache.hcatalog.pig.HCatLoader();

Specialized JOINS :

1. Replicated Join
2. Skewed Join
3. Merge Join

Replicated Join

Ø It works successfully if one of the data set is smaller in size which is to be fit in memory.

Ø It works very efficiently because smaller data set is copied into distributed cache which will be shared across to all the mappers in the cluster of machines. Also it implements join process at mapper itself in which reducer phase is avoided.

Ø According to the Pig documentation, a relation of size up to 100 MB can be used when the process has 1 GB of memory.

Ø A run-time error will be generated if not enough memory is available for loading the data.

Ø Replicated Join can be used in both inner and outer join. And it also supports for joining more than two tables.

Example

A_Big = LOAD 'emp.dat' USING PigStorage() AS (f1:int, f2:int, f3:int);

B_Small = LOAD 'salary.dat' USING PigStorage() AS (f1:int, f2:int, f3:int);

C = JOIN A_Big BY f1, B_Small BY f1 USING 'replicated';

Skewed Join

Ø Usually parallel join process will be harmed, if there are lots of data for a certain key, then data will not be evenly distributed across the reducers in which one of them will be stuck in processing the majority of data. Skewed join handles this case efficiently.

Sreeni Docs

Ø Skewed join computes a histogram of the key space and it uses this data to allocate reducers for a given key.

Ø Skewed Join can be used in both inner and outer join. And currently it only supports for joining two tables.

Ø The pig.skwedjoin.reduce.memusage Java parameter specifies the heap fraction available to reducers in order to perform this join. Setting a low value means more reducers will be used, yet the cost of copying the data across them will increase.

Ø Pig's developers claim to have good performance when setting it between 0.1 - 0.4.

Example

A_Big = LOAD 'emp.dat' USING PigStorage() AS (f1:int, f2:int, f3:int);

B_massive = LOAD 'salary.dat' USING PigStorage() AS (f1:int, f2:int, f3:int);

C = JOIN A_Big BY f1, B_massive BY f1 USING 'skewed';

Merge Join

Ø It works successfully if both the data sets are sorted(ascending order) by the same join key.

Ø It improves performance because join process takes place at mapper phase itself and it ignores two phases that are sort & Shuffle and reducer.

Ø Pig implements the merge join algorithm by selecting the left input of the join to be the input file for the map phase, and the right input of the join to be the side file. It then samples records from the right input to build an index that contains, for each sampled record, the key(s) the filename and the offset into the file the record begins at. This sampling is done in an initial map only job. A second or actual Map Reduce job is then initiated, with the left input as its input. Each map uses the index to seek to the appropriate record in the right input and begin doing the join.

Ø Merge join is only supported inner join.

Example

C = JOIN A BY f1, B BY f1 USING 'merge';

JSON:

Pig comes with a JSON loader. To load you use:

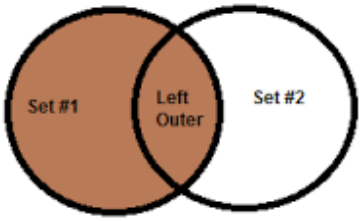
A = LOAD 'data.json' USING PigJsonLoader();

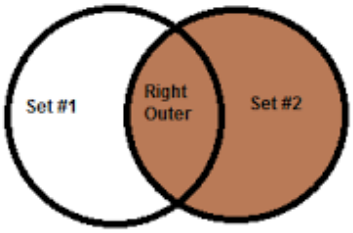
To store you can use:

STORE INTO 'output.json' USING PigJsonLoader();

JOINS:

Sreeni Docs





Left Outer:
Records from the first data set are included whether they have a match or not.Fields from the unmatched bag are set to null.

Userinfo_leftouter = join aa BY user left outer,bb BY user;

Right Outer:
The opposite to the left outer join. Records from the second data set are included no matter what. Fields from unmatched bag are set to null.

Userinfo_leftouter = join aa BY user right outer,bb BY user;

Full Outer:
Records from both sides are included.For unmatched records the fields from the other bag are set to NULL.
Userinfo_leftouter = join aa BY user full outer,bb BY user;

Sreen